

Metodología de Sistemas II

Trabajo Práctico 3 Observer

Farias, Gustavo

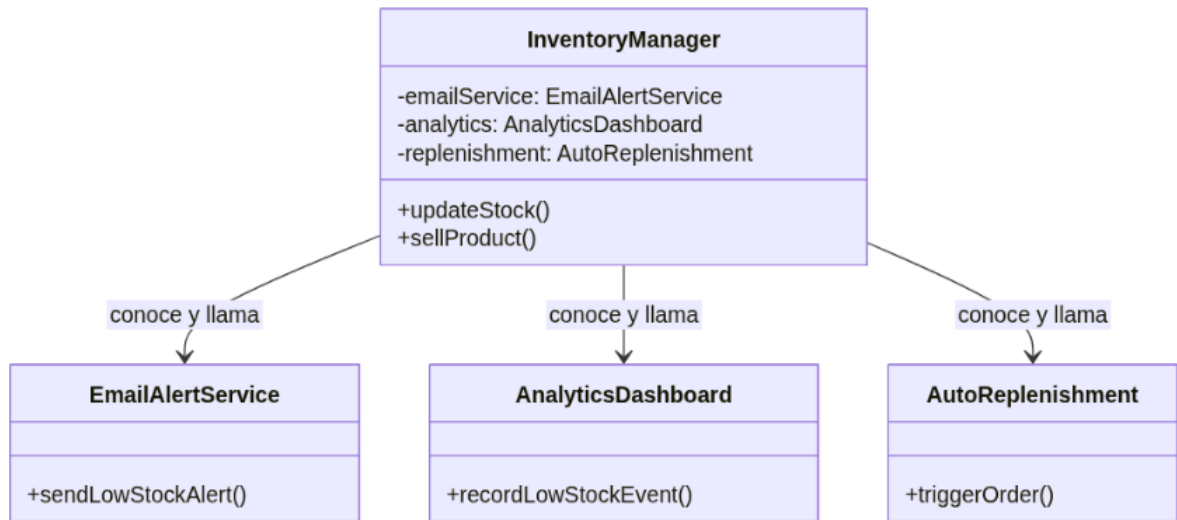
Matrícula: 101662

Comisión: M2025-13

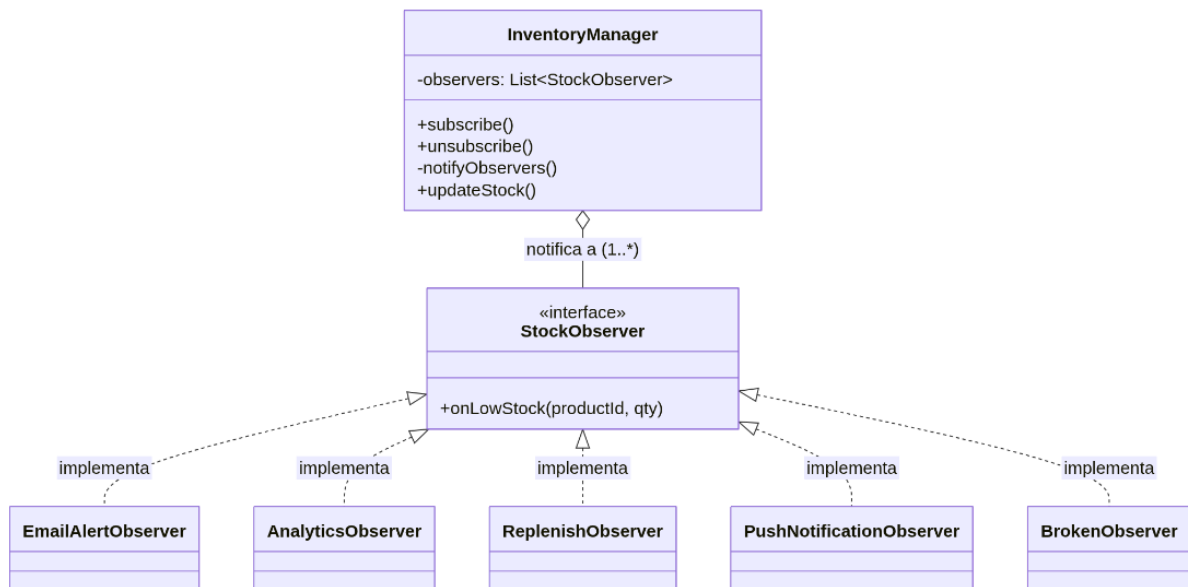
Repositorio GitHub: [Lucenear/Tecnicatura_Programacion_UTN](https://github.com/Lucenear/Tecnicatura_Programacion_UTN)

Diagrama UML

UML antes:



UML despues:



Codigo antes del Observer:

```
public class InventoryManager {
    private Map<String, Integer> stock = new HashMap<>();

    // Instancia directamente a los servicios concretos
    private EmailAlertService emailService;
    private AnalyticsDashboard analytics;
    private AutoReplenishment replenishment;
```

```

public InventoryManager() {
    this.emailService = new EmailAlertService();
    this.analytics = new AnalyticsDashboard();
    this.replenishment = new AutoReplenishment();
}

public void updateStock(String productId, int quantity) {
    stock.put(productId, quantity);
    if (quantity < 10) {
        emailService.sendLowStockAlert(productId, quantity);
        analytics.recordLowStockEvent(productId);
        replenishment.triggerOrder(productId, 100);
    }
}

public void sellProduct(String productId, int sold) {
    int current = stock.getDefault(productId, 0);
    int newQty = current - sold;
    stock.put(productId, newQty);
    if (newQty < 10) {
        emailService.sendLowStockAlert(productId, newQty);
        analytics.recordLowStockEvent(productId);
        replenishment.triggerOrder(productId, 100);
    }
}
}

class EmailAlertService {
    public void sendLowStockAlert(String productId, int qty) {
        System.out.println("[EMAIL] Stock bajo: " + productId + " (" + qty + ")");
    }
}

class AnalyticsDashboard {
    public void recordLowStockEvent(String productId) {
        System.out.println("[ANALYTICS] Evento registrado: " + productId);
    }
}

class AutoReplenishment {
    public void triggerOrder(String productId, int amount) {
        System.out.println("[REPLENISH] Orden creada para " + productId + " por " + amount + " unidades.");
    }
}

```

```
}  
}
```

Codigo despues del Observer:

Interfaz y Observadores concretos

```
public interface StockObserver {  
    void onLowStock(String productId, int quantity);  
}  
  
public class EmailAlertObserver implements StockObserver {  
    @Override  
    public void onLowStock(String productId, int quantity) {  
        System.out.println("[EMAIL] Alerta enviada: Stock bajo de  
" + productId + " (" + quantity + ")");  
    }  
}  
  
public class AnalyticsObserver implements StockObserver {  
    @Override  
    public void onLowStock(String productId, int quantity) {  
        System.out.println("[ANALYTICS] Evento registrado: " +  
productId);  
    }  
}  
  
public class ReplenishObserver implements StockObserver {  
    @Override  
    public void onLowStock(String productId, int quantity) {  
        System.out.println("[REPLENISH] Orden de reposición creada  
para " + productId + " (100 unidades)");  
    }  
}  
  
public class PushNotificationObserver implements StockObserver {  
    @Override  
    public void onLowStock(String productId, int quantity) {  
        System.out.println("[PUSH] Notificación enviada sobre " +  
productId);  
    }  
}
```

```
// Test e observador roto
public class BrokenObserver implements StockObserver {
    @Override
    public void onLowStock(String productId, int quantity) {
        System.out.println("[BROKEN] Intentando procesar...");
        throw new RuntimeException("Error de red simulado en
BrokenObserver");
    }
}
```

Notificador refactorizado

```
public class InventoryManager {
    private Map<String, Integer> stock = new HashMap<>();

    // El manager solo conoce la interfaz
    private List<StockObserver> observers = new ArrayList<>();

    public void subscribe(StockObserver observer) {
        observers.add(observer);
    }

    public void unsubscribe(StockObserver observer) {
        observers.remove(observer);
    }

    // Try catch para que no corte la cadena
    private void notifyObservers(String productId, int quantity) {
        for (StockObserver observer : observers) {
            try {
                observer.onLowStock(productId, quantity);
            } catch (Exception e) {
                System.err.println("[ERROR] Observer fallo: " +
e.getMessage() + " - Continuando...");
            }
        }
    }

    public void updateStock(String productId, int quantity) {
        stock.put(productId, quantity);
        if (quantity < 10) {
            notifyObservers(productId, quantity);
        }
    }
}
```

```

    }

    public void sellProduct(String productId, int sold) {
        int current = stock.getDefault(productId, 0);
        int newQty = current - sold;
        stock.put(productId, newQty);
        if (newQty < 10) {
            notifyObservers(productId, newQty);
        }
    }
}

```

Main (test del Observer roto)

```

public class Main {
    public static void main(String[] args) {
        InventoryManager manager = new InventoryManager();

        // suscribo los observers
        manager.subscribe(new EmailAlertObserver());
        manager.subscribe(new AnalyticsObserver());
        manager.subscribe(new BrokenObserver());           // Rompe
        manager.subscribe(new ReplenishObserver());
        manager.subscribe(new PushNotificationObserver()); //

        Nuevo

        System.out.println("--- Actualizando stock a 5 (debe
disparar alertas) ---");
        manager.updateStock("PROD-001", 5);
    }
}

```

Descarte las llamadas directas porque acoplaban el inventario a las notificaciones: sumar canales exigía modificar el manager. Elegí Observer para invertir esta dependencia: el manager solo emite eventos a una interfaz genérica. Así, agregar suscriptores no toca la lógica de inventario, y el try-catch dentro del loop aísla los fallos sin interrumpir al resto de la cadena.